Now you can proceed to complete the Solidity code for the Ballot contract based on the details specified in the contract diagram.

3.2.9 Putting it all together

The complete code in Solidity is given in listing 3.5. Only the functions are shown because we already reviewed the data and modifier definitions in listings 3.1 through 3.4. Earlier, you saw only the templates of the functions; here, you see the completed code. Note that the Phase component is included in this code to illustrate the state transitions, the FSM-based design of dynamics, and the use of modifier-based validation.

Listing 3.5 Solution with modifier `validPhase` (BallotV3.sol)

```
// include listing 3.1 data here

   // modifiers
   modifier validPhase(Phase reqPhase)
    { require(state == reqPhase);
      _;
    }

    constructor (uint numProposals) public  {
        chairperson = msg.sender;
        voters[chairperson].weight = 2; // weight 2 for testing purposes
        for (uint prop = 0; prop < numProposals; prop ++)
            proposals.push(Proposal(0));
        state = Phase.Regs; // change Phase to Regs

    }

    function changeState(Phase x) public {                              ❶
        if (msg.sender != chairperson) {revert();}                      ❷
        if (x < state ) revert();
        state = x;
    }


    function register(address voter) public validPhase(Phase.Regs) {    ❸
        if (msg.sender != chairperson || voters[voter].voted) revert(); ❷
        voters[voter].weight = 1;
        voters[voter].voted = false;

    }
```